
A BRAVE NEW CURRICULUM - SOLUCIÓN GUÍA

Universidad de Bogotá Jorge Tadeo Lozano

Alvarado Becerra Ludwig - Franco Calderón Alejandro
Estructuras de datos - 2026-1S

Índice

1. Interpretación del problema, entrada y salida de datos	1
1.1. Objetivo	1
1.2. Entradad de datos esperada	1
1.3. Salida de datos esperada	1
2. Solución conceptual	1
3. Código modelo	2
3.1. Lectura de datos	2
3.2. Votos totales para cada candidato	2

1. Interpretación del problema, entrada y salida de datos

1.1. Objetivo

Dada una cadena de paréntesis, verificar si esta es valida comprobando el emparejamiento de los paréntesis.

1.2. Entradas de datos esperadas

Se nos diferencian cadenas de paréntesis en cada línea enviada al programa, el programa debe terminar cuando se encuentra una línea que sea 0

```
((()))  
{({[[[({})]]])}}}  
[]{()  
{}  
0
```

Algorítmicamente, se debe parar el algoritmo cuando la entrada sea '0'.

1.3. Salidas de datos esperadas

Por cada cadena de paréntesis, imprimir YES en caso de que la cadena sea valida, si no lo es, imprimir NO. Tener en cuenta que la salida debe ser en mayúsculas.

2. Solución conceptual

En el contexto de un compilador, es fundamental verificar que los paréntesis, corchetes y llaves estén correctamente balanceados. Es decir:

- Cada símbolo de apertura debe tener su correspondiente cierre.
- El orden de cierre debe ser correcto.

La idea clave es pensar en que **el último paréntesis debe ser el primero en cerrarse**. Esta idea encaja con el principio de funcionamiento de un *stack* (pila), donde el último elemento que ingresa es el primero en salir. Las operaciones básicas de un *stack* incluyen:

- `push(x)`, guardar `x` en la pila.
- `is_empty()`, retorna `true` si no hay elementos en la pila, en caso contrario devuelve `false`.
- `pop()`, saca (y retorna) el último elemento que fue guardado en la pila.

Lo ideal es tener dos estructuras de datos para guardar la información:

- `string` para guardar la entrada de datos.

- **stack**, usado para almacenar los paréntesis abiertos que se encuentren.

Teniendo en cuenta la lógica de programación, los pasos a seguir con el algoritmo¹ es:

1. Recorrer el **string** de izquierda a derecha.
2. Si se encuentra un símbolo de apertura (, {, [. Entonces, agregar al **stack** usando **push()**.
3. Si se encuentra un símbolo de cierre), },]:
 - Si la pila está vacía, entonces imprimir NO.
 - Usar **pop()** y verificar si coincide con el paréntesis de apertura, si sí, seguir, si no, imprimir NO.
4. Después de haber iterado sobre todo el **string**, verificar si está vacío el **stack**, si sí, imprimir YES, de lo contrario, imprimir NO.

3. Código modelo

Se invita a que el estudiante intente implementar la solución únicamente con el algoritmo descrito en los pasos antes de ver el código modelo de la solución.

3.1. Lectura de datos

Se debe iniciar el programa leyendo datos como tipo **str** y guardar la información dentro de una variable **s**. Esta acción se debe repetir hasta que en algún momento **s** sea 0, en este momento el programa se debe de detener. Para hacer la acción de repetir indefinidamente una serie de acciones mientras se cumple una condición, se puede utilizar un ciclo tipo **while**

```
1 s = input()
2
3 while (s != '0'):
4     ...
5     s = input()
```

Donde ... es la lógica que se implementará en próximos pasos

3.2. Votos totales para cada candidato

¹Se recomienda leer el algoritmo en pasos e intentar pasarlo a código antes de ver la solución del código modelo



UTADEO

UNIVERSIDAD DE BOGOTÁ JORGE TADEO LOZANO